MANAGING SECURITY AND USER DATA IN TERRAFORM

HOW TO INSTALL TERRAFORM ON LOCAL (UBUNTU)

1. wget -O - https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
2. echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(grep -oP '(?<=UBUNTU_CODENAME=).*' /etc/os-release || lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list
3. sudo apt update && sudo apt install terraform
4. aws configure
   access_key: <>
   secret_key: <>
   region: ap-south-1
5. terraform --version

TERRAFORM LIFECYCLE

1. terraform init

- Initializes the Terraform working directory.
- Downloads required providers and plugins.
- Prepares the project for Terraform operations.

2. terraform plan

- Creates an execution plan or blueprint of the infrastructure.
- Shows what resources will be created, modified, or deleted.
- Helps review changes before applying them.

3. terraform fmt

- Automatically formats Terraform configuration files.
- Fixes indentation, spacing, and alignment.
- Maintains standard and readable code structure.
- Useful for clean and professional Infrastructure as Code (IaC).

4. terraform validate

- Validates whether the Terraform configuration file is correct or not.
- Checks syntax and configuration errors.
- Ensures the code is properly written.

5. terraform apply

- Creates or updates the infrastructure resources defined in the file.
- Executes the changes shown in the plan.
- Deploys the actual infrastructure.

6. terraform destroy

- Deletes all resources created by Terraform.
- Removes the entire infrastructure setup.
- Used for cleanup or environment removal.

CREATING 1ST EC2 INSTANCE USING TERRAFORM

1. **Provider block**
   Setup connection between the cloud provider and terraform
2. **Resource block**
   Write the data (arguments) to create a resource like s3bucket, ec2instances etc.

3. To set-up profile in ubuntu

3. aws configure --profile config
   1. access_key:
   2. secret_key:
   3. region: ap-south-1
4. aws configure list-profiles

ON VS-CODE

1.provider.tf

```
provider "aws" {
  region = "ap-south-1"
  profile = "config"
}
```

2.ec2_instance.tf

```
resource "aws_instance" "my_instance" {
  ami           = "ami-0685bcc683dadb6b9"
  instance_type = "t3.micro"
  key_name      = "ubuntu"
  tags = {
    Name = "webserver"
  }
}
```

3.git add .
4.git commit -m “ec2.tf”
5.git push

ON-UBUNTU LOCAL

1. git clone https://github.com/mayurpandit25/terraform-b30.git
2. cd terraform-b30
3. terraform init
4. terraform plan
5. terraform validate
6. terraform apply --auto-approve
7. terraform destroy --auto-approve

FILES THAT ARE CREATED WITH TERRAFORM LIFECYCLE

TERRAFORM INIT

1. .terraform/

1. Hidden folder automatically created by Terraform
2. Downloads and stores provider plugins like AWS, Azure, Kubernetes
3. Stores downloaded Terraform modules
4. Stores backend initialization data
5. Helps Terraform run infrastructure code properly

2. .Terraform.lock.hcl

1. Locks the provider versions used in the project
2. Ensures every user uses the same provider versions
3. Prevents version conflicts between team members
4. Maintains consistent Terraform execution
5. Improves stable and reliable deployments

TERRAFORM APPLY

1. terraform.tfstate

1. Stores the current state of infrastructure
2. Keeps track of all created resources
3. Connects Terraform code with real cloud resources
4. Stores resource details like IDs and configurations
5. Helps Terraform identify infrastructure changes

2. terraform.tfstate.backup

1. Stores backup of the previous state file

2. Automatically created before updating state
3. Helps recover old infrastructure state if needed
4. Protects against state file corruption
5. Improves infrastructure recovery and safety
